

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ARPITA GANGADHAR PATTAR (1BM24CS053)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by ARPITA GANGADHAR PATTAR (**1BM24CS053**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Syed Akram
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-12
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	13-16
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	17-19
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	20-22
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	23-24
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	25-27
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	28-31 32-35
9	Implement Fractional Knapsack using Greedy technique.	36-38
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	39-41
11	Implement "N-Queens Problem" using Backtracking.	42-45
12	Leetcode	46-51

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
```

```
void topologicalSort(int n, int adj[100][100])
```

```
{  
    int queue[100];  
    int indegree[100];  
    int front = 0, rear = -1;  
    for (int i = 0; i < n; i++)  
    {  
        indegree[i] = 0;  
    }  
    for (int i = 0; i < n; i++)  
    {  
        for (int j = 0; j < n; j++)  
        {  
            if (adj[i][j] == 1)  
            {  
                indegree[j]++;  
            }  
        }  
    }  
    for (int i = 0; i < n; i++)  
    {  
        if (indegree[i] == 0)  
        {  
            queue[++rear] = i;  
        }  
    }
```

```

    }

    printf("Topological Order: ");
    while (front <= rear)
    {
        int node = queue[front++];
        printf("%d ", node);

        for (int i = 0; i < n; i++)
        {
            if (adj[node][i] == 1)
            {
                indegree[i]--;

                if (indegree[i] == 0)
                {
                    queue[++rear] = i;
                }
            }
        }
    }
    printf("\n");
}

int main()
{
    int n;
    int adj[100][100];

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

```

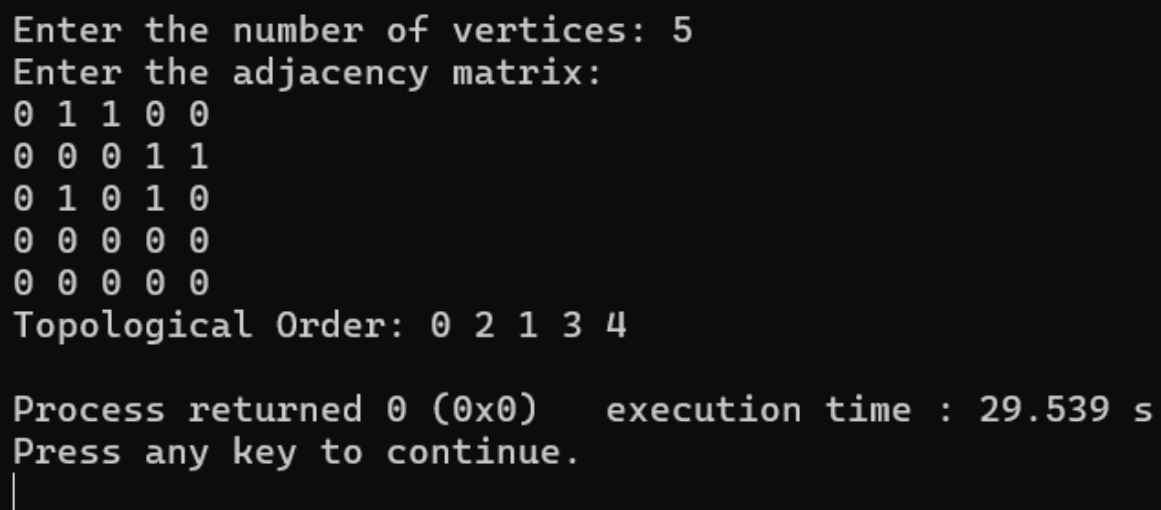
```
printf("Enter the adjacency matrix:\n");

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        scanf("%d", &adj[i][j]);
    }
}

topologicalSort(n, adj);

return 0;
}
```

OUTPUT:



```
Enter the number of vertices: 5
Enter the adjacency matrix:
0 1 1 0 0
0 0 0 1 1
0 1 0 1 0
0 0 0 0 0
0 0 0 0 0
Topological Order: 0 2 1 3 4

Process returned 0 (0x0)    execution time : 29.539 s
Press any key to continue.
|
```

Observation:

Date: __/__/__
Page: __

```

* Topological Sort

void topologicalSort (int v, int adj[][v])
{
    int indegree[v], queue[v];
    int front = 0, rear = -1;

    for (int i = 0; i < v; i++)
        indegree[i] = 0;
    for (int i = 0; i < v; i++)
    {
        for (int j = 0; j < v; j++)
        {
            if (adj[i][j] == 1)
                indegree[j]++;
        }
    }
    for (int i = 0; i < v; i++)
        if (indegree[i] == 0)
            queue[++rear] = i;

    printf("\n Topological order:");

    while (front <= rear)
    {
        int node = queue[front++];
        printf("%d ", node);
        for (int i = 0; i < v; i++)
        {
            if (adj[node][i] == 1)
            {
                indegree[i]--;
                if (indegree[i] == 0)
                    queue[++rear] = i;
            }
        }
    }
}

```

Date: __/__/__
Page: __

```

queue[++rear] = i;
}

}

}

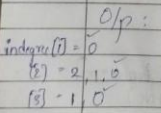
int main()
{
    int v, adj[10][10];
    printf("Enter the number of vertices:");
    scanf("%d", &v);

    printf("\n Enter the adjacency matrix:");

    for (int i = 0; i < v; i++)
        for (int j = 0; j < v; j++)
            scanf("%d", &adj[i][j]);

    topologicalSort(v, adj);
    return 0;
}

```

O/p:  adj:

0	1	1	0	0
0	0	0	1	1
0	1	0	1	0
0	0	0	0	0
0	0	0	0	0

Indegree:
 (1) = 0
 (2) = 0
 (3) = 0
 (4) = 0
 (5) = 1

Topological Order: 1 3 2 4 5

Lab Program 2: Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
```

```
int LR = 1, RL = 0;
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int fact(int n)
```

```
{
```

```
    int result = 1;
```

```
    for (int i = 1; i <= n; i++)
```

```
        result = result * i;
```

```
    return result;
```

```
}
```

```
int permutation(int a[], int dir[], int n)
```

```
{
```

```
    int prev = 0, mobile = 0, pos = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (i != 0 && dir[a[i] - 1] == RL)
```

```
        {
```

```
            if (a[i] > a[i - 1] && a[i] > prev)
```

```
            {
```

```

        mobile = a[i];
        prev = mobile;
    }
}
if (i != n - 1 && dir[a[i] - 1] == LR)
{
    if (a[i] > a[i + 1] && a[i] > prev)
    {
        mobile = a[i];
        prev = mobile;
    }
}
}
for (int i = 0; i < n; i++)
{
    if (a[i] == mobile)
        pos = i + 1;
}
if (dir[a[pos - 1] - 1] == RL)
    swap(&a[pos - 1], &a[pos - 2]);
else if (dir[a[pos - 1] - 1] == LR)
    swap(&a[pos], &a[pos - 1]);

for (int i = 0; i < n; i++)
{
    if (a[i] > mobile)
    {

```

```

        if (dir[a[i] - 1] == LR)
            dir[a[i] - 1] = RL;
        else if (dir[a[i] - 1] == RL)
            dir[a[i] - 1] = LR;
    }
}

for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
printf("\n");
return 0;
}

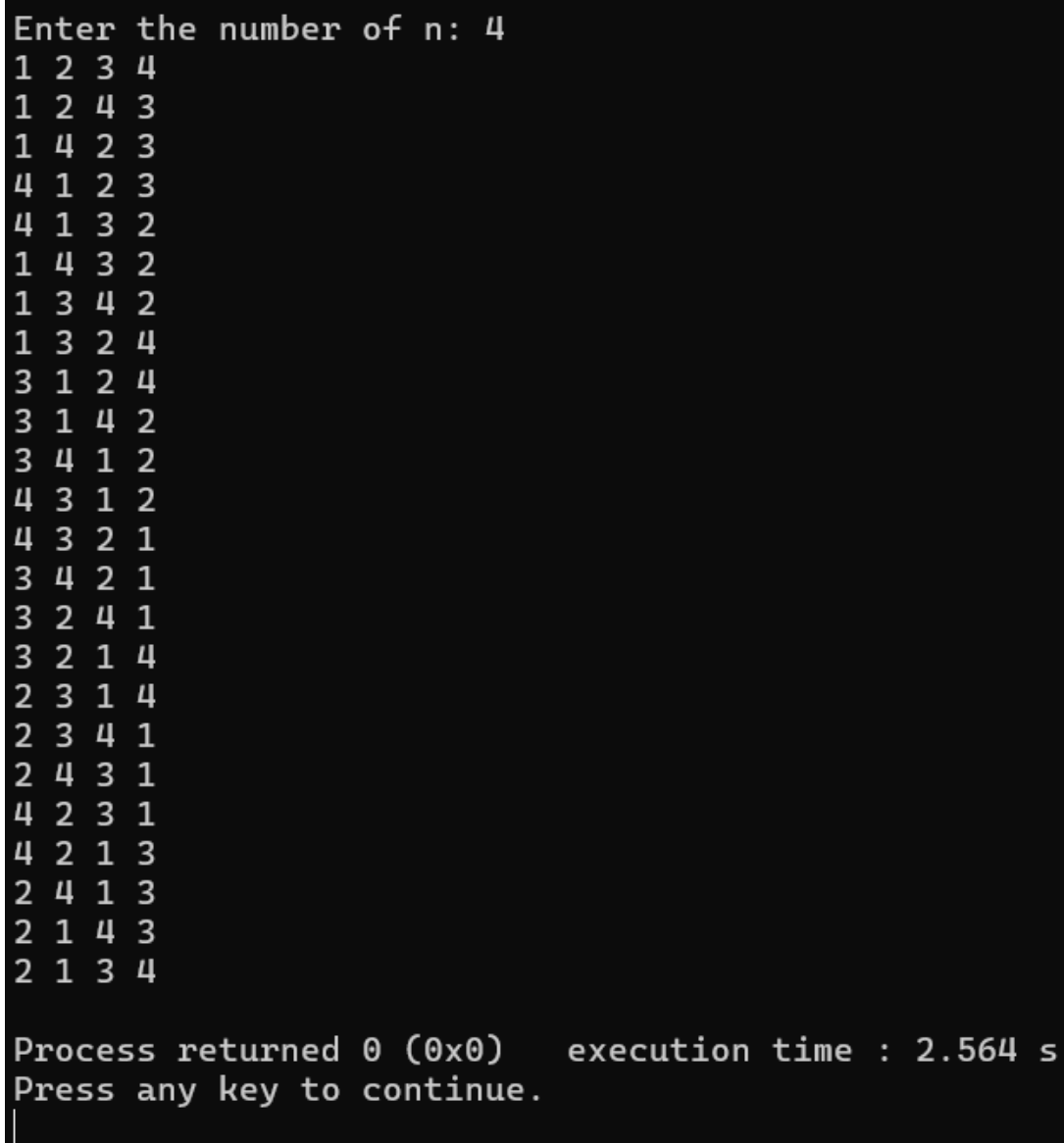
int main()
{
    int n;
    printf("Enter the number of n: ");
    scanf("%d", &n);
    int a[n], dir[n];

    for (int i = 0; i < n; i++)
    {
        a[i] = i + 1;
        printf("%d ", a[i]);
    }
    printf("\n");

```

```
for (int i = 0; i < n; i++)  
    dir[i] = RL;  
for (int i = 1; i < fact(n); i++)  
    permutation(a, dir, n);  
  
return 0;  
}
```

Ouput:



```
Enter the number of n: 4  
1 2 3 4  
1 2 4 3  
1 4 2 3  
4 1 2 3  
4 1 3 2  
1 4 3 2  
1 3 4 2  
1 3 2 4  
3 1 2 4  
3 1 4 2  
3 4 1 2  
4 3 1 2  
4 3 2 1  
3 4 2 1  
3 2 4 1  
3 2 1 4  
2 3 1 4  
2 3 4 1  
2 4 3 1  
4 2 3 1  
4 2 1 3  
2 4 1 3  
2 1 4 3  
2 1 3 4  
  
Process returned 0 (0x0)   execution time : 2.564 s  
Press any key to continue.  
|
```

Observation:

Program :

```
#include <stdio.h>

int LR=1, RL=0;

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int fact(int n)
{
    int result = 1;
    for (int i=1; i<=n; i++)
        result = result * i;
    return result;
}

int permutation(int a[], int dir[], int n)
{
    int prev = a, mobile = a, pos;
    for (int i=0; i<n; i++)
    {
        if (dir[a[i]-1] == RL || i==0)
        {
            if (a[i] > a[i-1] || a[i] > prev)
            {
                mobile = a[i];
                prev = mobile;
            }
        }
        if (dir[a[i]-1] == LR || i==n-1)
        {
            if (a[i] < a[i-1] || a[i] < prev)
            {
                mobile = a[i];
                prev = mobile;
            }
        }
    }
}
```

```

{
    if (a[i] > a[i-1] || a[i] > prev)
    {
        mobile = a[i];
        prev = mobile;
    }
}
}

for (int i=0; i<n; i++)
{
    if (a[i] == mobile)
        pos = i+1;
}

if (dir[a[pos-1]-1] == RL)
    swap(&a[pos-1], &a[pos-2]);
else if (dir[a[pos-1]-1] == LR)
    swap(&a[pos], &a[pos-1]);

for (int i=0; i<n; i++)
{
    if (a[i] > mobile)
    {
        if (dir[a[i]-1] == LR)
            dir[a[i]-1] = RL;
        else if (dir[a[i]-1] == RL)
            dir[a[i]-1] = LR;
    }
}

for (int i=0; i<n; i++)
    printf("%d ", a[i]);
}
}

```

```

int main()
{
    int n;
    printf("Enter the number of n: ");
    scanf("%d", &n);

    int a[n], dir[n];
    for (int i=0; i<n; i++)
    {
        a[i] = i+1;
        printf("%d ", a[i]);
    }
    for (int i=0; i<n; i++)
        dir[i] = RL;

    for (int i=1; i<=fact(n); i++)
        permutation(a, dir, n);
    return 0;
}

```

Lab Program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
```

```
void MergeSort(int a[],int b[],int m,int n)
```

```
{
```

```
    int i=0,j=0,k=0,c[100];
```

```
    while(i<m && j<n)
```

```
    {
```

```
        if(a[i]<b[j])
```

```
        {
```

```
            c[k]=a[i];
```

```
            k++;
```

```
            i++;
```

```
        }
```

```
    else
```

```
    {
```

```
        c[k]=b[j];
```

```
        k++;
```

```
        j++ ;
```

```
    }
```

```
}
```

```
while(i<m)
```

```
{
```

```
    c[k]=a[i];
```

```
    k++;
```

```
    i++;
```

```

    }
    while(j<n)
    {
        c[k]=b[j];
        k++;
        j++;
    }
    printf("Merged Array:");
    for(int i=0;i<(m+n);i++)
        printf("%d\t",c[i]);
}

int main()
{
    int n,m,a[100],b[100];
    printf("Enter the number of elements in First array:");
    scanf("%d",&m);
    printf("Enter the number of elements of first array:");
    for(int i=0;i<m;i++)
        scanf("%d",&a[i]);

    printf("Enter the number of elements in Second array:");
    scanf("%d",&n);
    printf("Enter the number of elements of first array:");
    for(int i=0;i<n;i++)
        scanf("%d",&b[i]);

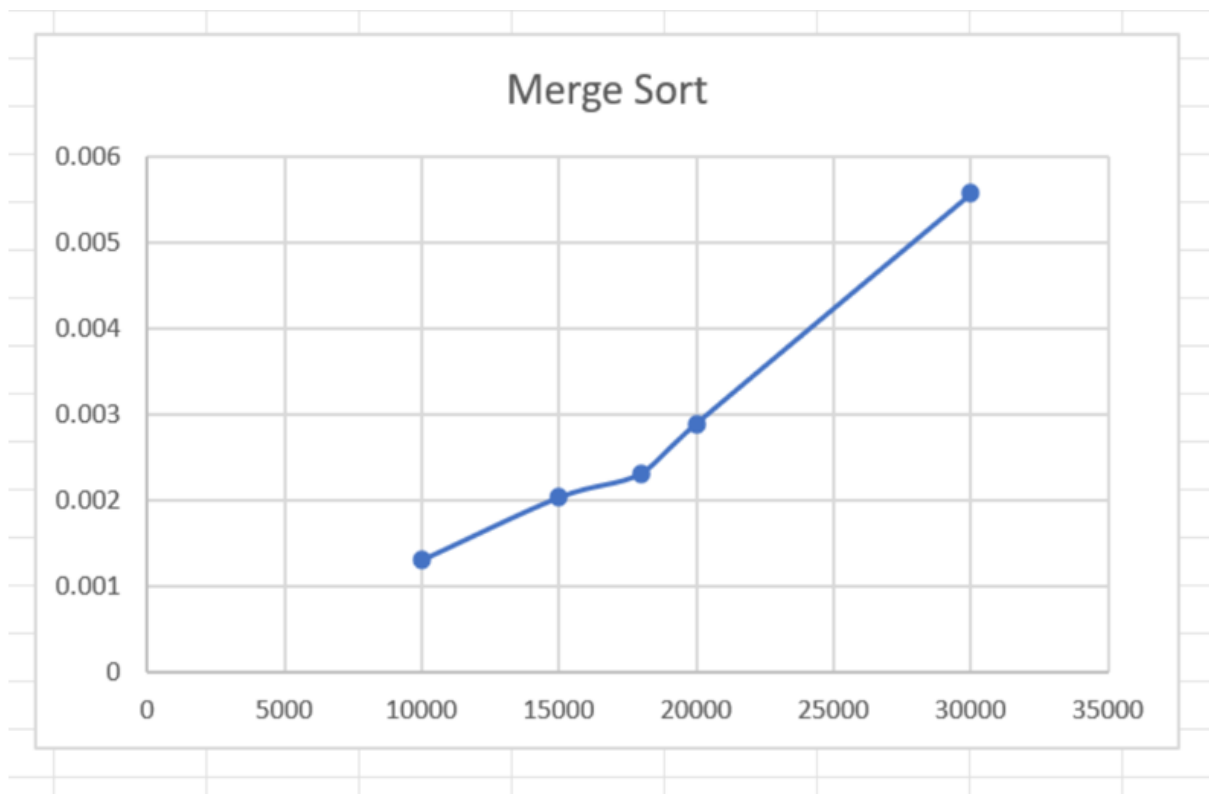
```

```
MergeSort(a,b,m,n);  
  
return 0;  
  
}
```

Output:

```
Enter the number of elements in First array:5  
Enter the number of elements of first array:1 3 5 7 9  
Enter the number of elements in Second array:5  
Enter the number of elements of first array:2 4 6 8 10  
Merged Array:1 2 3 4 5 6 7 8 9 10  
Process returned 0 (0x0) execution time : 15.473 s  
Press any key to continue.  
|
```

Graph:



Observation:

Date: / /
Page:

*** Merge Sort**

Algorithm

1. Start
2. Take 2 sorted arrays as inputs
3. Compare between elements of array 1 & array 2
if array 1 element > array 2
put it in new array
i++ & k++
else put array 2 element in new array
j++ & k++
4. Check if first array is empty
if not empty
shift all elements to new array
i++ & k++
5. Check if array 2 is empty.
if not empty
shift all elements to new array
j++ & k++

Stop.

Program:

```
#include <stdio.h>
int main()
{
    int low, mid, high, n, m;
    int a[50], b[50];

    printf("Enter the size of arrays: ");
    scanf("%d %d", &m, &n);
    printf("Enter the array 1 elements:");
    for (int i=0; i<m; i++)
        scanf("%d", &a[i]);
```

Date: / /
Page:

```
    printf("Enter array 2 elements:");
    for (int j=0; j<n; j++)
        scanf("%d", &b[j]);

    merge sort (m, n, a, b);
}

void mergesort (int m, int n, int a[], int b[])
{
    int i=0, j=0, c[100];
    while (i<m & j<n)
    {
        if (a[i] < b[j])
        {
            c[k] = a[i];
            i++;
            k++;
        }
        else
        {
            c[k] = b[j];
            j++;
            k++;
        }
    }
    while (i<m)
    {
        c[k] = a[i];
        i++;
        k++;
    }
    while (j<n)
    {
        c[k] = b[j];
        j++;
        k++;
    }
}
```

Date: / /
Page:

```
    printf("Merged array:");
    for (int i=0; i<(m+n); i++)
        printf("%d\t", c[i]);
    return 0;
}
```

O/p: m=10, n=10
Enter array1 elements: 1 3 5 7 9 11 13 15 17 19
Enter array2 elements: 2 4 6 8 10 12 14 16 18
Merged array: 0 1 2 3 4 5 6 7 8 9 10 11
12 13 14 15 16 17 18 19
Execution time: 0.00100.

Lab Program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
int partition(int a[] ,int low,int high)
```

```
{
```

```
    int i,j,temp,key;
```

```
    i=low+1;
```

```
    j=high;
```

```
    key=a[low];
```

```
    while(1)
```

```
    {
```

```
        while(a[i]<=key && i<=high)
```

```
            i++;
```

```
        while(a[j]>key)
```

```
            j--;
```

```
        if(i<j){
```

```
            temp=a[i];
```

```
            a[i]=a[j];
```

```
            a[j]=temp;
```

```
        }
```

```
        else{
```

```
            temp=a[low];
```

```
            a[low]=a[j];
```

```
            a[j]=temp;
```

```
            return j;
```

```
        }
```

```
    }
```

```
}
```

```

void QuickSort(int a[],int low,int high)
{
    int j;
    if(low<high)
    {
        j=partition(a,low,high);
        QuickSort(a,low,j-1);
        QuickSort(a,j+1,high);
    }
}

int main()
{
    int n,a[100];
    printf("Enter the number of elements in the array:");
    scanf("%d",&n);
    printf("Enter the array elements:");
    for(int i=0;i<n;i++)
        scanf("%d",&a[i]);
    QuickSort(a,0,n-1);

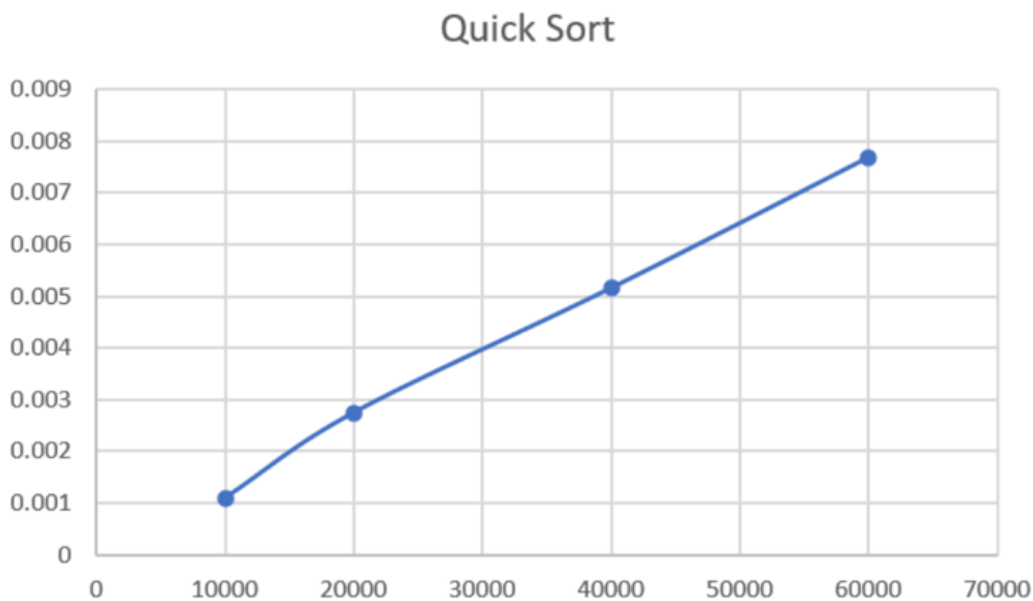
    printf("Sorted Array:");
    for(int i=0;i<n;i++)
        printf("%d\t",a[i]);
    return 0;
}

```

Output:

```
Enter the number of elements in the array:6
Enter the array elements:12 5 8 23 56 89
Sorted Array:5 8 12 23 56 89
Process returned 0 (0x0) execution time : 11.668 s
Press any key to continue.
```

Graph:



Observation:

```
Program:
#include <stdio.h>

int QuickSort(int a[], int low, int high)
{
    int j;
    if (low < high)
    {
        j = partition(a, low, high);
        quickSort(a, low, j-1);
        quickSort(a, j+1, high);
    }
}

int partition(int a[], int low, int high)
{
    int i, j, temp, key;
    i = low;
    j = high;
    key = a[low];
    while (1)
    {
        while (a[i] <= key && i <= high)
            i++;
        while (a[j] > key)
            j--;
        if (i < j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        else
        {
            temp = a[low];
            a[low] = a[j];
            a[j] = temp;
        }
    }
}
```

```
int main()
{
    int n;
    printf("Enter the number of elements:");
    scanf("%d", &n);

    int a[n];
    printf("Enter the elements:");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n-1);
    printf("Sorted array:");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

o/p:
Enter the number of elements: 5
Enter the elements:
10
2
8
9
15
Sorted array: 2 8 9 10 15

Lab Program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
void heapify(int a[],int n,int p)
```

```
{
```

```
    int c,item;
```

```
    item=a[p];
```

```
    c=2*p+1;
```

```
    while(c<n)
```

```
    {
```

```
        if(c+1<n)
```

```
        {
```

```
            if(a[c]<a[c+1])
```

```
                c++;
```

```
        }
```

```
        if(item<a[c])
```

```
        {
```

```
            a[p]=a[c];
```

```
            p=c;
```

```
            c=2*p+1;
```

```
        }
```

```
        else
```

```
            break;
```

```
    }
```

```
    a[p]=item;
```

```
}
```

```
void Build_heap(int a[],int n)
{
    for(int i=(n-1)/2;i>=0;i--)
        heapify(a,n,i);
}
```

```
void heapsort(int a[],int n)
{
    int i,temp;
    Build_heap(a,n);

    for(int i=n-1;i>0;i--)
    {
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
    printf("Sorted Array:");
    for(int i=0;i<n;i++)
        printf("%d\t",a[i]);
}
```

```
int main()
{
    int a[100],n,i;
    printf("Enter the number of elements:");
    scanf("%d",&n);
    printf("Enter the array elements: ");
    for(i=0;i<n;i++)
```

```

scanf("%d",&a[i]);

heapsort(a,n);

return 0;

}

```

Output:

```

Enter the number of elements:8
Enter the array elements: 13 8 5 2 1 19 26 15
Sorted Array:1  2    5      8      13      15      19      26
Process returned 0 (0x0)   execution time : 24.709 s
Press any key to continue.
|

```

Observation:

Program:

```

#include <stdio.h>
#include <time.h>

void heapify (int a[], int n, int p)
{
    int c, item;
    item = a[p];
    c = 2*p+1;
    while (c < n)
    {
        if (c+1 < n)
            if (a[c] < a[c+1])
                c++;
        if (item < a[c])
        {
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }
        else
            break;
    }
    a[p] = item;
}

void build_heap (int a[], int n)
{
    for (int i = (n-1)/2; i >= 0; i--)
        heapify (a, n, i);
}

```

```

void heapsort (int a[], int n)
{
    int i, temp;
    build_heap (a, n);
    for (i = n-1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify (a, i, 0);
    }
    printf ("Sorted array:");
    for (i = 0; i < n; i++)
        printf ("%d\t", a[i]);
}

int main()
{
    clock_t start, end;
    int a[100], n, i;

    printf ("Enter number of elements:");
    scanf ("%d", &n);
    printf ("Enter array elements:\n");
    for (i = 0; i < n; i++)
        scanf ("%d", &a[i]);

    start = clock();
    heapsort (a, n);
    end = clock();
    double time_taken = ((double)(end-start))
        / CLOCKS_PER_SEC;
}

```

```

printf ("In Execution time: %f\n",
        time_taken);
return 0;
}

Output:
Enter no. of elements: 8
Enter array elements:
13 8 5 2 1 19 26 15
Sorted array:
1 2 5 8 13 15 19 26
Execution time: 0.001000

```

Lab Program 6: Implement 0/1 Knapsack problem using dynamic programming.

```
#include<stdio.h>
```

```
int max(int a,int b)
```

```
{  
    if(a>b)  
        return a;  
    return b;  
}
```

```
int main()
```

```
{  
    int p[100],w[100],M,n,table[100][100];  
    printf("Enter the number of items:");  
    scanf("%d",&n);  
    printf("Enter the maximum capacity of Knapsack:");  
    scanf("%d",&M);  
    for(int i=1;i<=n;i++){  
        printf("Enter the weight and profit of item %d:",i);  
        scanf("%d %d",&w[i],&p[i]);  
    }  
    for(int i=0;i<=n;i++){  
        for(int j=0;j<=M;j++){  
            if(i==0 || j==0) {  
                table[i][j]=0;  
            }  
            else if(j<w[i]) {  
                table[i][j]=table[i-1][j];  
            }  
        }  
    }  
}
```

```

else {
    table[i][j]=max(table[i-1][j],p[i]+table[i-1][j-w[i]]);
}
}

printf("Maximum profit made:%d",table[n][M]);

return 0;
}

```

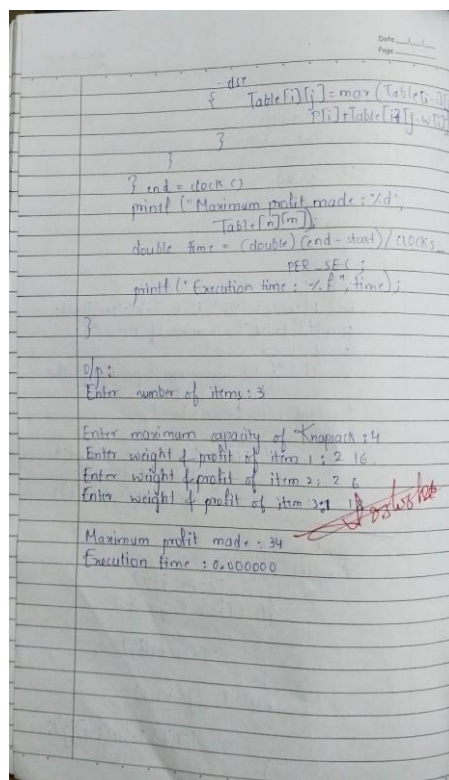
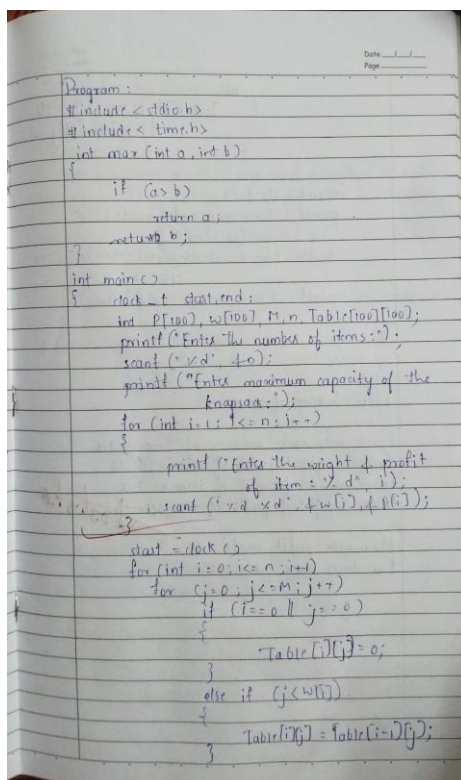
Output:

```

Enter the number of items:3
Enter the maximum capacity of Knapsack:50
Enter the weight and profit of item 1:10 60
Enter the weight and profit of item 2:20 100
Enter the weight and profit of item 3:30 120
Maximum profit made:220
Process returned 0 (0x0)    execution time : 16.977 s
Press any key to continue.
|

```

Observation:



Lab Program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>
```

```
#define INF 99
```

```
void Floyd's(int a[10][10],int n)
```

```
{ for(int k=0;k<n;k++)
```

```
{
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
for(int j=0;j<n;j++)
```

```
if(a[i][k]+a[k][j]<a[i][j])
```

```
a[i][j]=a[i][k]+a[k][j];
```

```
}
```

```
}
```

```
printf("Shortest Distance Matrix:\n");
```

```
for(int i=0;i<n;i++)
```

```
{
```

```
for(int j=0;j<n;j++)
```

```
if(a[i][j]==INF)
```

```
printf("INF");
```

```
else
```

```
printf("%d ",a[i][j]);
```

```
printf("\n");
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
int i,j,k,n;
```

```

int cost[10][10],a[10][10];

printf("Enter the number of Vertices:");

scanf("%d",&n);

printf("Enter the cost matrix:\n");

for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        scanf("%d",&cost[i][j]);

        a[i][j]=cost[i][j];
    }
}

Floyds(a,n);

return 0;
}

```

Output:

```

Enter the number of Vertices:5
Enter the cost matrix:
0 4 99 5 99
99 0 1 99 6
2 99 0 3 99
99 99 1 0 2
1 99 99 4 0
Shortest Distance Matrix:
0 4 5 5 7
3 0 1 4 6
2 6 0 3 5
3 7 1 0 2
1 5 5 4 0

Process returned 0 (0x0)    execution time : 36.980 s
Press any key to continue.
|

```

Observation:

Date: / /
Page:

```

* Floyd's Algorithm

#include <stdio.h>
#include <time.h>
#define INF 99

void Floyd (int A[][10], int n)
{
    int i, j, k;
    for (k=0; k<n; k++)
        for (i=0; i<n; i++)
            for (j=0; j<n; j++)
                if (A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];

    printf("\n Shortest distance matrix:");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
        {
            if (A[i][j] == INF)
                printf("INF");
            else
                printf("%d", A[i][j]);
        }
    printf("\n");
}

int main ()
{
    int i, j, k, n;
    int cost[10][10], A[10][10];
    printf("Enter num of vertices:");
    scanf ("%d", &n);
    printf("Enter the cost matrix:\n");
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf ("%d", &cost[i][j]);

```

Date: / /
Page:

```

        A[i][j] = cost[i][j];
    }
    clock = t_start, end
    start = clock ();
    Floyd (A, n);
    end = clock ();
    double time = (double)(end - start) /
        CLOCKS_PER_SEC;
    printf("Execution time: %f", time);
    return 0;
}

o/p:
Enter the number of vertices: 3
Enter the cost matrix:
0 4 99 5 99
99 0 1 99 6
2 99 0 3 99
99 99 1 0 2
1 99 99 4 0

Shortest distance matrix:
0 4 5 5 2
3 0 1 4 6
2 6 0 3 5
3 3 1 0 2
1 5 5 4 0

Execution time: 0.000000

```

works

Lab Program 8:(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define MAX INT_MAX
```

```
int minKey(int key[], int set[], int n)
```

```
{
    int min = MAX, min_id;
    for (int i = 0; i < n; i++)
    {
        if (set[i] == 0 && key[i] < min)
        {
            min = key[i];
            min_id = i;
        }
    }
    return min_id;
}
```

```
int main()
```

```
{
    int n, parent[100], key[100], set[100], graph[100][100];
    int mincost = 0;
    clock_t start, end;
    printf("Enter the number of vertices in the graph: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
```

```

        scanf("%d", &graph[i][j]);
    }
}
start = clock();

for (int i = 0; i < n; i++)
{
    key[i] = MAX;
    set[i] = 0;
}
key[0] = 0;
parent[0] = -1;

for (int count = 0; count < n - 1; count++)
{
    int i = minKey(key, set, n);
    set[i] = 1;
    for (int j = 0; j < n; j++)
    {
        if (graph[i][j] &&
            set[j] == 0 &&
            graph[i][j] < key[j])
        {
            parent[j] = i;
            key[j] = graph[i][j];
        }
    }
}
end = clock();
printf("\nEdge\tWeight\n");
for (int i = 1; i < n; i++)
{
    printf("%d - %d\t%d\n",parent[i],i, graph[parent[i]][i]);
}

```

```

        mincost += graph[parent[i]][i];
    }
    printf("\nMinimum cost is %d\n", mincost);
    double time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Execution time: %f\n", time);
    return 0;
}

```

Output:

```

Enter the number of vertices in the graph: 5
Enter the adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5

Minimum cost is 16
Execution time: 0.000000

Process returned 0 (0x0)    execution time : 50.840 s
Press any key to continue.

```

Observation:

```

Date: / /
Page:

* Prim's Algorithm

Algorithm
step 1: Start
step 2: Create a set to keep track of vertices in MST
step 3: Assign a key value to all vertices & initialise
to infinity
step 4: key[0] = 0, parent[0] = -1
step 5: for (i = 0; i < n; i++)
{
    u = min cost of edge not yet selected in
    MST
    MST[u] = 1
    for (int j = 0; j < n; j++)
    {
        if (graph[i][j] & & MST[j] == 0
            & graph[i][j] < key[j])
        {
            parent[j] = i;
            key[j] = graph[i][j];
        }
    }
}

step 6: return mincost

Program:
#include <stdio.h>
#include <time.h>

int minKey (int key[], int set[], int n)
{
    int min = MAX, min_idx;
    for (int v = 0; v < n; v++)
    {
        if (set[v] == 0 & key[v] < min)
        {
            min = key[v]; min_idx = v;
        }
    }
}

```

```

Date: / /
Page:

return min_idx;
}

int main()
{
    int n, parent[100], key[100], set[100], graph[100][100];
    int mincost = 0;
    clock_t start, end;
    printf("Enter the number of vertices in the graph:");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    start = clock();
    for (int i = 0; i < n; i++)
    {
        key[i] = MAX;
        set[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < n; count++)
    {
        int i = minKey (key, set, n);
        set[i] = 1;
        for (int j = 0; j < n; j++)
        {
            if (graph[i][j] & & set[j] == 0 &
                graph[i][j] < key[j])
            {
                parent[j] = i;
                key[j] = graph[i][j];
            }
        }
    }

    end = clock();
}

```

```

Date: / /
Page:

printf("Edge Weight\n");
for (int i = 1; i < n; i++)
{
    printf("%d - %d \t %d\n", 0, i, graph[0][i]);
    parent[i] = 0, graph[parent[i]][i];
    mincost += graph[parent[i]][i];
}

printf("\n Minimum cost is %d", mincost);
double time = (double) (end - start) /
    CLOCKS_PER_SEC;
printf("Execution time : %f", time);
return 0;
}

o/p:
Enter the number of vertices of the graph: 5
Enter the adjacency matrix:
0 2 0 6 0
2 0 5 0 0
0 5 0 0 0
0 8 0 0 9
0 5 7 9 0

Edge Weight
0-1 2
1-2 5
0-3 6
1-4 5
Minimum cost: 16
Execution time: 0.00000

```

Lab Program 8:(b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>
```

```
typedef struct
```

```
{
```

```
    int u,v,w;
```

```
}Edge;
```

```
Edge edges[100],result[100];
```

```
int parent[100];
```

```
int find(int i)
```

```
{
```

```
    while(parent[i]!=i)
```

```
    {
```

```
        i=parent[i];
```

```
    }
```

```
    return i;
```

```
}
```

```
void unionSet(int i,int j)
```

```
{
```

```
    parent[i]=j;
```

```
}
```

```
void sortEdge(int e)
```

```
{
```

```
    int i,j;
```

```
    Edge temp;
```

```
    for(i=0;i<e-1;i++){
```

```
        for(j=0;j<e-1-i;j++){
```

```
            {
```

```

        if(edges[j].w>edges[j+1].w){
            temp=edges[j];
            edges[j]=edges[j+1];
            edges[j+1]=temp;
        }
    }
}

int kruskals(int n,int e)
{
    int count=0;
    int cost=0;
    int u,v;
    for(int i=0;i<n;i++){
        parent[i]=i;
    }
    sortEdge(e);
    for(int i=0;i<e&& count<n-1;i++){
        {
            u=find(edges[i].u);
            v=find(edges[i].v);
            if(u!=v){
                result[count++]=edges[i];
                cost+=edges[i].w;
                unionSet(u,v);
            }
        }
    }
    printf("Minimum Spanning Tree:\n");
    for(int i=0;i<count;i++){
        printf("%d---%d = %d\n",result[i].u,result[i].v,result[i].w);
    }
}

```

```

        return cost;
    }

int main()
{
    int n,e;
    printf("Enter the number of vertices:");
    scanf("%d",&n);
    printf("Enter the number of edges:");
    scanf("%d",&e);
    printf("Enter the edges(u,v,weight):\n");
    for(int i=0;i<e;i++){
        scanf("%d %d %d",&edges[i].u,&edges[i].v,&edges[i].w);
    }
    int cost=kruskals(n,e);
    printf("Total Cost:%d\n",cost);
    return 0;
}

```

Output:

```

Enter the number of vertices:7
Enter the number of edges:9
Enter the edges(u,v,weight):
1 2 4
1 3 28
2 5 26
3 6 12
3 4 8
4 5 22
4 7 16
6 7 6
5 7 20
Minimum Spanning Tree:
1---2 = 4
6---7 = 6
3---4 = 8
3---6 = 12
5---7 = 20
2---5 = 26
Total Cost:76

Process returned 0 (0x0)   execution time : 55.094 s
Press any key to continue.
|

```

Observation:

Date: ___/___/___
Page: ___

*** Kruskal's Algorithm**

Step 1: Start
 Step 2: Initialize each vertex as a set
 Step 3: Initialize $i=0$, $mincost=0$
 Step 4: while ($i < n-1$ & edges left to be considered)
 find mincost of edge (u,v)
 $j = find(u)$
 $k = find(v)$
 if ($j \neq k$)
 $i++$
 $mincost += cost(u,v)$
 $t[i,1] = 0$
 $t[i,2] = u$
 union(j,k)
 }

Step 5: if ($i = n-1$)
 return "No MST"
 else
 return MST

Step 6: Stop

*** Program**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct
{
    int u, v, w;
} Edge;
```

Date: ___/___/___
Page: ___

```
Edge edges[100], result[100];
int parent[100];

int find(int i)
{
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionset(int i, int j)
{
    parent[i] = j;
}

void costEdge(int e)
{
    int i, j;
    Edge temp;
    for (i = 0; i < e-1; i++)
    {
        for (j = 0; j < e-i-1; j++)
        {
            if (edges[j].w > edges[j+1].w)
            {
                temp = edges[j];
                edges[j] = edges[j+1];
                edges[j+1] = temp;
            }
        }
    }
}

int kruskals(int n, int e)
{
    int count = 0;
    int cost = 0;
    int u, v;
```

Date: ___/___/___
Page: ___

```
for (int i = 0; i < n; i++)
    parent[i] = i;
costEdge(e);
for (int i = 0; i < e & count < n-1; i++)
{
    u = find(edges[i].u);
    v = find(edges[i].v);
    if (u != v)
    {
        result[count++] = edges[i];
        cost += edges[i].w;
        unionset(u, v);
    }
}

printf("Minimum Spanning Tree:\n");
for (int i = 0; i < count; i++)
    printf("%d -- %d == %d\n", result[i].u, result[i].v, result[i].w);

return cost;
}

int main()
{
    int n, e;
    printf("Enter the number of vertices:");
    scanf("%d", &n);
    printf("Enter the number of edges:");
    scanf("%d", &e);
    printf("Enter the edges (u, v, weight):\n");
    for (int i = 0; i < e; i++)
    {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }
}
```

Date: ___/___/___
Page: ___

```
int cost = kruskals(n, e);
printf("Total cost = %d\n", cost);
return 0;
}
```

O/p:
 Enter the number of vertices: 7
 Enter the number of edges: 9
 Enter the edges (u, v, weight):

1	2	4
1	3	28
2	5	26
3	6	12
3	4	8
4	5	22
4	7	16
6	7	6
5	7	20

Minimum Spanning Tree:

```
1 --- 2 == 4
6 --- 7 == 6
3 --- 4 == 8
3 --- 6 == 12
5 --- 7 == 20
2 --- 5 == 26
Total cost == 76
```

Abhishek

Lab Program 9: Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

int main() {
    int i, j, n, m;
    int p[50], w[50];
    float x[50];
    int t1, t2;

    printf("Enter the number of objects: ");
    scanf("%d", &n);

    printf("Enter the weights of all objects:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &w[i]);
    }

    printf("Enter the profit of all objects:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &p[i]);
    }

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if ((float)p[i] / w[i] < (float)p[j] / w[j]) {

                t1 = p[i];
                p[i] = p[j];
                p[j] = t1;

                t2 = w[i];
                w[i] = w[j];
                w[j] = t2;
            }
        }
    }

    printf("Enter the maximum capacity: ");
    scanf("%d", &m);

    for (i = 0; i < n; i++) {
        x[i] = 0.0;
    }

    int rem_cap = m;

    for (i = 0; i < n; i++) {
```

```

        if (w[i] > rem_cap) {
            break;
        }
        x[i] = 1.0;
        rem_cap -= w[i];
    }

    if (i < n) {
        x[i] = (float)rem_cap / w[i];
    }

    float sum = 0;
    for (i = 0; i < n; i++) {
        sum = sum + (p[i] * x[i]);
    }

    printf("Total profit : %f\n", sum);

    return 0;
}

```

Output:

```

Enter the number of objects: 3
Enter the weights of all objects:
18 15 10
Enter the profit of all objects:
25 24 15
Enter the maximum capacity: 20
Total profit : 31.500000

Process returned 0 (0x0)   execution time : 13.103 s
Press any key to continue.

```

Observation:

*** Knapsack problem using greedy method**

Algorithm:

- 1) Start
- 2) Read n, p (profit) & weight of all 'n' objects.
- 3) Sort them in decreasing order of $\frac{p[i]}{w[i]}$
- 4) for $i=1$ to n { $x[i] = 0$;
remaining_capacity = m;
for $i=1$ to n do
{
if ($w[i] > \text{remaining_capacity}$)
break;
remaining_capacity = $w[i]$;
if ($i \leq n$)
then $x[i] = \frac{\text{remaining_capacity}}{w[i]}$;
int profit = 0;
for $i=1$ to n
profit += $p[i] * x[i]$;
return profit;

Program:

```
#include <stdio.h>
int i, j, p[50], w[50], n, m;
float x[50];

int main()
{
    printf("Enter the number of objects:");
    scanf("%d", &n);
    printf("Enter the weights of all objects:");
```

```
for (i=0; i<n; i++)
    scanf("%d", &w[i]);
printf("Enter the profit of all objects:");
for (i=0; i<n; i++)
    scanf("%d", &p[i]);
for (i=0; i<n-1; i++)
{
    for (j=i+1; j<n; j++)
    {
        if ((float)(p[i])/w[i] < (float)(p[j])/w[j])
        {
            int t1 = p[i];
            p[i] = p[j];
            p[j] = t1;
            int t2 = w[i];
            w[i] = w[j];
            w[j] = t2;
        }
    }
}

printf("Enter the maximum capacity:");
scanf("%d", &m);
for (i=0; i<n; i++)
{
    x[i] = 0;
}

int rem_cap = m;
for (i=0; i<n; i++)
{
    if (w[i] > rem_cap)
        break;
}
```

```
x[i] = 1;
rem_cap -= w[i];
}
if (i < n)
{
    x[i] = (float)(rem_cap) / w[i];
}
float sum = 0;
for (i=0; i<n; i++)
{
    sum = sum + (p[i] * x[i]);
}
printf("Total profit: %.2f", sum);
}
```

O/p:

```
Enter the number of objects: 3
Enter the weights of all objects: 13
15
0
Enter the profits associated with the weights: 23
24
15
Enter the maximum capacity: 20
Total profit: 31.500000
```

Time-complexity:

$$T(n) = \sum_{i=1}^n \sum_{j=1}^n 1$$

$$= \sum_{i=1}^n (n-1+1)$$

$$= (n-1) \sum_{i=1}^n 1 = (n-1)(n-2+1)$$

$$= (n-1)(n-1) = (n-1)^2 = n^2 - 2n + 1$$

$$\therefore T(n) = O(n^2)$$

Lab Program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define n 5
```

```
int minDist(int dist[], int visited[]) {
```

```
    int min = 99999, min_index = -1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (visited[i] == 0 && dist[i] <= min) {
```

```
            min = dist[i];
```

```
            min_index = i;
```

```
        }
```

```
    }
```

```
    return min_index;
```

```
}
```

```
int dijkstra(int weight[n][n], int src) {
```

```
    int dist[n];
```

```
    int visited[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        dist[i] = 99999;
```

```
        visited[i] = 0;
```

```
    }
```

```
    dist[src] = 0;
```

```
    for (int count = 0; count < n - 1; count++) {
```

```
        int u = minDist(dist, visited);
```

```
        visited[u] = 1;
```

```
        for (int v = 0; v < n; v++) {
```

```
            if (!visited[v] && weight[u][v] != 0 && dist[u] != 99999) {
```

```
                int new_dist = dist[u] + weight[u][v];
```

```

        if (new_dist < dist[v]) {
            dist[v] = new_dist;
        }
    }
}

printf("Vertex \t Distance from source\n");
for (int i = 0; i < n; i++) {
    printf("%d \t\t %d\n", i + 1, dist[i]);
}
return 0;
}

int main() {
    int i, j;
    int graph[n][n];
    clock_t start, end;

    printf("Enter the weight matrix (%dx%d):\n", n, n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int source = 0;
    start = clock();
    dijkstra(graph, source);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Time taken: %f seconds\n", time_taken);

    return 0;
}

```

Output:

Enter the weight matrix (5x5):

```
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
```

Vertex Distance from source

```
1 0
2 10
3 50
4 30
5 60
```

Time taken: 0.001000 seconds

Process returned 0 (0x0) execution time : 124.393 s
Press any key to continue.

Observation:

```
#include <stdio.h>
#include <limits.h>

#define INF 99999
#define n 5

int minDist(int dist[], int visited[])
{
    int min = INF, min_index;
    for (int i = 0; i < n; i++)
    {
        if (!visited[i] && dist[i] < min)
        {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

int dijkstra(int graph[n][n], int src)
{
    int dist[n];
    int visited[n];
    for (int i = 0; i < n; i++)
    {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count = 0; count < n - 1; count++)
    {
        int u = minDist(dist, visited);
        visited[u] = 1;
    }
}
```

```
for (int v = 0; v < n; v++)
{
    if (!visited[v] && graph[u][v] < INF)
    {
        new_dist = dist[u] + graph[u][v];
        if (new_dist < dist[v])
        {
            dist[v] = new_dist;
        }
    }
}

printf("Vertex\tDistance from source\n");
for (int i = 0; i < n; i++)
    printf("%d\t\t%d\n", i, dist[i]);
return 0;

int main()
{
    int i, j, weight[n][n];
    clock_t start, end;
    printf("Enter the weight matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &weight[i][j]);

    int source = 0;
    start = clock();
    dijkstra(graph, source);
    end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Time taken is %d second", time_taken);
    return 0;
}
```

```
for (int v = 0; v < n; v++)
{
    if (!visited[v] && graph[u][v] < INF)
    {
        new_dist = dist[u] + graph[u][v];
        if (new_dist < dist[v])
        {
            dist[v] = new_dist;
        }
    }
}

printf("Vertex\tDistance from source\n");
for (int i = 0; i < n; i++)
    printf("%d\t\t%d\n", i, dist[i]);
return 0;

int main()
{
    int i, j, weight[n][n];
    clock_t start, end;
    printf("Enter the weight matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &weight[i][j]);

    int source = 0;
    start = clock();
    dijkstra(graph, source);
    end = clock();

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Time taken is %d second", time_taken);
    return 0;
}
```

Graph:

```

graph LR
    0 ---|10| 1
    0 ---|30| 2
    0 ---|100| 4
    1 ---|50| 2
    2 ---|20| 3
    3 ---|60| 4

```

Time taken: 0 seconds.

Algorithm

- Start
- Initialize dist[] with infinity (i.e. 99999) from 0 to n
- Initialize visited[] with 0 from 0 to n
- Set dist[source] = 0
- for i to n-1 do
 u = min(dist, visited)
 mark visited[u] = 1;

Lab Program 11: Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdbool.h>

int solutionCount = 0;

bool isSafe(int N, int board[N][N], int row, int col) {
    int i, j;
    for (i = 0; i < col; i++) {
        if (board[row][i]) {
            return false;
        }
    }
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j]) {
            return false;
        }
    }
    for (i = row, j = col; j >= 0 && i < N; i++, j--) {
        if (board[i][j]) {
            return false;
        }
    }
    return true;
}

void printSolution(int N, int board[N][N]) {
    printf("Solution %d:\n", ++solutionCount);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
    }
}
```

```

        printf("\n");
    }
    printf("\n");
}

void solveNQueenRecursive(int N, int board[N][N], int col) {
    if (col >= N) {
        printSolution(N, board);
        return;
    }
    for (int i = 0; i < N; i++) {
        if (isSafe(N, board, i, col)) {
            board[i][col] = 1;
            solveNQueenRecursive(N, board, col + 1);
            board[i][col] = 0;
        }
    }
}

void solveNQ(int N) {
    int board[N][N];

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }

    solveNQueenRecursive(N, board, 0);

    if (solutionCount == 0) {
        printf("Solution does not exist\n");
    } else {
        printf("Total solutions = %d\n", solutionCount);
    }
}

```

```

    }
}

int main() {
    int N;
    printf("Enter value of N: ");
    scanf("%d", &N);
    if (N <= 0) {
        printf("Enter a positive number.\n");
        return 0;
    }
    solveNQ(N);
    return 0;
}

```

Output:

```

Enter value of N: 4
Solution 1:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Solution 2:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Total solutions = 2

Process returned 0 (0x0)    execution time : 14.530 s
Press any key to continue.

```

Observation:

*** N-Queens Problem**

1. Start

2. Place the queen at first cell of 1st row (1st column)

3. place Move to next row, place the queen in first cell. if:

- it is in same column as previous queen
- it move to next column, check all the
- end; if get move to next column
- (or) move to previous row move the queen to next cell

4. Repeat '3' till all the rows are filled.

5. Stop.

*** Program**

```
#include <stdio.h>
#include <stdlib.h>

int solutionCount = 0;

bool isSafe(int N, int board[N][N], int row, int col)
{
    int i, j;
    for (i = 0; i < row; i++)
        if (board[i][col])
            return false;
    for (j = col; j < N; j++)
        if (board[row][j])
            return false;
    for (j = col; j > 0; j--)
        if (board[row][j])
            return false;
}
```

```
return false;

for (i = row; i < N; i++)
    for (j = col; j > 0; j--)
        if (board[i][j])
            return false;
return true;

void printSolution (int N, int board[N][N])
{
    printf("Solution: %d\n", ++solutionCount);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            printf("%d ", board[i][j]);
    printf("\n");
}

void solveNQueen (int N, int board[N][N], int col)
{
    if (col >= N)
        printSolution(N, board);
        return;
    for (int i = 0; i < N; i++)
        if (isSafe(N, board, i, col))
        {
            board[i][col] = 1;
            solveNQueen(N, board, col+1);
            board[i][col] = 0;
        }
}

void solveNQ (int N)
{
    int board[N][N];
}
```

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        board[i][j] = 0;

solveNQueen(N, board, 0);
if (solutionCount == 0)
    printf("Solution does not exist\n");
else
    printf("Total solutions = %d\n", solutionCount);
}

int main()
{
    int N;
    printf("Enter value of N:");
    scanf("%d", &N);

    if (N <= 0)
        printf("Enter a positive number: \n");
        return 0;
    solveNQueen(N);
    return 0;
}

o/p: Enter value of N: 4
Solution 1: 0 0 1 0
            1 0 0 0
            0 0 0 1
            0 1 0 0
```

Solution 2:

```
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

Total solutions = 2

Let Code

1. Topological sort - Course Schedule

```
bool canFinish (int numCourses, int** prerequisites,
               int prerequisitesSize, int* prerequisitesColSize)
{
    int indegree[numCourses];
    int queue[numCourses], front, rear, count;
    front = 0, rear = 0, count = 0;

    for (int i = 0; i < prerequisitesSize; i++)
        int course = prerequisites[i][0],
            prereq = prerequisites[i][1];
            indegree[course]++;

    for (int i = 0; i < numCourses; i++)
        if (indegree[i] == 0)
            queue[rear++] = i;

    while (front < rear)
    {
        int currentCourse = queue[front++];
        count++;
        for (int i = 0; i < prerequisitesSize; i++)
            if (prerequisites[i][1] == currentCourse)
                indegree[prerequisites[i][0]]--;
                if (indegree[prerequisites[i][0]] == 0)
                    queue[rear++] = prerequisites[i][0];
    }

    return count == numCourses;
}
```

Leetcode Program 1: Leet Code exercises on topological sorting(207).

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
    int inDegree[1000] = {0};
    int queue[1000], front, rear, count;
    front = rear = count = 0;

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        inDegree[course]++;
    }
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i;
        }
    }
    while (front < rear) {
        int currCourse = queue[front++];
        count++;
        for (int i = 0; i < prerequisitesSize; i++) {
            if (prerequisites[i][1] == currCourse) {
                int depCourse = prerequisites[i][0];
                inDegree[depCourse]--;

                if (inDegree[depCourse] == 0) {
                    queue[rear++] = depCourse;
                }
            }
        }
    }
    return count == numCourses;
}
```

Output:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Accepted Runtime: 0 ms

☐ Case 1 ☒ Case 2

Input

numCourses =
2

prerequisites =
[[1,0],[0,1]]

Output

false

Expected

false

Leetcode Program 2: Leet Code exercise on Knapsack problem using dynamic Programming(801).

```
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {  
    int x = 0, y = 1;  
  
    for (int i = 1; i < nums1Size; i++) {  
        int a = x, b = y;  
        if (nums1[i - 1] >= nums1[i] || nums2[i - 1] >= nums2[i]) {  
            x = b;  
            y = a + 1;  
        }  
        else {  
            y = b + 1;  
  
            if (nums1[i - 1] < nums2[i] &&  
                nums2[i - 1] < nums1[i])  
                x = (x < b) ? x : b;  
            y = (y < a + 1) ? y : (a + 1);  
        }  
    }  
    return (x < y) ? x : y;  
}
```

Output:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[0,3,5,8,9]

nums2 =
[2,1,4,6,9]

Output

1

Expected

1

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums1 =
[1,3,5,4]

nums2 =
[1,2,3,7]

Output

1

Expected

1

Leetcode Program 3: Leet Code exercise on Floyd's algorithm(287).

```
int findDuplicate(int* nums, int numsSize)
```

```
{  
    int slow1, fast, slow2;  
    slow1=fast=slow2=0;  
  
    while(true)  
    {  
        slow1=nums[slow1];  
        fast=nums[nums[fast]];  
        if(slow1==fast)  
        {  
            break;  
        }  
    }  
}
```

```
while(true)  
{  
    slow1=nums[slow1];  
    slow2=nums[slow2];  
    if(slow1==slow2)  
    {  
        return slow2;  
    }  
}  
}
```

Output:

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[1,3,4,2,2]
```

Output

```
2
```

Expected

```
2
```

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[3,1,3,4,2]
```

Output

```
3
```

Expected

```
3
```

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[3,3,3,3,3]
```

Output

```
3
```

Expected

```
3
```

